

Binary Search Tree Algorithms:

Topic 1:

BST(Binary Search Tree) Search

- Start from the root.
- Compare the searching element with root, if less than root, then recursively call left subtree, else recursively call right subtree.
- If the element to search is found anywhere, return true, else return false.

1. Recursive Algorithm to search a given key in a given BST

Input: Pointer to the root node of the Tree, Key

Output: Returns the pointer to the node, if key is found
Returns NULL, if the key is not found

Steps:

```
node* search(node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->key == key)
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}
```

2. Iterative Algorithm to search a given key in a given BST

Input: Pointer to the root node of the Tree, Key

Output: Returns true, if key is found
Returns false, if the key is not found

Steps:

```
bool iterativeSearch(Node* root, int key)
{
    // Traverse until root reaches to dead end
    while (root != NULL) {
        // pass right subtree as new tree
        if (key > root->data)
            root = root->right;
        // pass left subtree as new tree
        else if (key < root->data)
            root = root->left;
        else
            return true; // if the key is found return 1
    }
    return false;
}
```

Topic 2:

BST(Binary Search Tree) Insert:

Approach:

As we all know, a new key is always inserted at the leaf node. so we start searching the key from the root till we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node with the given value, while searching if the value of current node is greater then the given value then move to the left , else move to right.

Follow the steps mentioned below to implement the idea:

- Start from the root and run a loop until a null pointer is reached.
- Keep the previous pointer of the current node stored.
- If the current node is null then create and insert the new node there and make it as one of the children of the parent/previous node depending on its value.
- If the value of the current node is less than the new value then move to the right child of the current node else move to the left child.

Here is the code to create the new node with left, right pointers set to NULL and data is set to the Key field.

```
Node* newNode(int data)
{
    Node* temp = new Node;
    temp->key = data;
    temp->left = NULL;
    temp->right = NULL;
    return temp;
}
```

3. Iterative algorithm to Insert the node in the BST

Input: Pointer to the root node of the Tree, Key

Output: Returns the pointer where new node is inserted

Steps:

```
Node* insert(Node* root, int key)
```

```
{ // Create a new Node containing the new element
```

```
    Node* newnode = newNode(key);
```

```
    //Pointer to start traversing from root and traverses downward path to search
```

```
    //where the new node to be inserted
```

```
    Node* x = root;
```

```
    // Pointer y maintains the trailing pointer of x
```

```
    Node* y = NULL;
```

```
    while (x != NULL) {
```

```
        y = x;
```

```
        if (key < x->key)
```

```
            x = x->left;
```

```
        else
```

```
            x = x->right;
```

```
    }
```

```
    // If the root is NULL i.e the tree is empty, the new node is the root node
```

```
    if (y == NULL)
```

```
        y = newnode;
```

```
    // If the new key is less than the leaf node key, assign the new node to be its left child
```

```
    else if (key < y->key)
```

```
        y->left = newnode;
```

```
    // else assign the new node its right child
```

```
    else
```

```
        y->right = newnode;
```

```
    // Returns the pointer where the new node is inserted
```

```
    return y;
```

```
}
```

Topic 3:

BST Delete

Given a binary search tree and a node of the binary search tree, the task is to delete the node from the Binary Search tree Iteratively. Here are the three cases that arise while performing a delete operation on a BST:

Case 1: Node to be deleted is a leaf node. Directly delete the node from the tree.



Case 2: Node to be deleted is an internal node with two children. Copy the contents of the inorder successor of the node to be deleted and delete the inorder successor. The inorder successor can be found by finding the minimum element in the right subtree of the node. $\text{inorderSuccessor}(10) = 11$.



Case 3: Node to be deleted is an internal node with one child. For this case, delete the node and move its child up to take its place.



The intuition behind deleting the inorder successor in Case 2 is that the inorder successor of a node with two children will always be greater than all elements in the left subtree of the node since it is the smallest node in the right subtree of the node and the inorder successor of the node will always be smaller than all other nodes in the right subtree of the node.

This preserves the BST property that all nodes in the left subtree of a given node are smaller than the given node and all nodes in the right subtree of the given node are greater than the given node.

4. Iterative algorithm to delete the node in the BST

Input: Pointer to the root node of the Tree, Key

Output: Returns the pointer to the root of the BST after deletion of key

Steps:

```
Node* deleteIterative(Node* root, int key)
```

```
{
```

```
    Node* curr = root;
```

```
    Node* prev = NULL;
```

/*Check if the key is actually present in the BST. The variable prev points to the parent of the key to be deleted.*/

```
    while (curr != NULL && curr->data != key) {
```

```
        prev = curr;
```

```
        if (key < curr->data)
```

```
            curr = curr->left;
```

```
        else
```

```
            curr = curr->right;
```

```
    }
```

```
    if (curr == NULL) {
```

```
        print("Key not found\n")
```

```
        return root; }
```

/* Case 1 and 3 discussed above. Check if the node to be deleted has at most one child.*/

if (curr->left == NULL || curr->right == NULL) {

// newCurr will replace the node to be deleted.

Node* newCurr;

// if the left child does not exist.

if (curr->left == NULL)

newCurr = curr->right;

else

newCurr = curr->left;

// Check if the node to be deleted is the root.

if (prev == NULL)

return newCurr;

/* check if the node to be deleted is prev's left or right child and then replace this with newCurr*/

if (curr == prev->left)

prev->left = newCurr;

else

prev->right = newCurr;

// free memory of the node to be deleted.

free(curr);

}

// Case 2 discussed above:

//Node to be deleted has two children.

else {

Node* p = NULL;

Node* temp;

// Compute the inorder successor

temp = curr->right;

while (temp->left != NULL) {

p = temp;

temp = temp->left;

}

/*check if the parent of the inorder successor is the curr or not(i.e. curr= the node which has the same data as the given data by the user to be deleted). if it isn't, then make the left child of its parent equal to the inorder successor's right child.*/

if (p != NULL)

p->left = temp->right;

/*if the inorder successor was the curr (i.e. curr = the node which has the same data as the given data by the user to be deleted), then make the right child of the node to be deleted equal to the right child of the inorder successor.*/

else

curr->right = temp->right;

curr->data = temp->data;

free(temp);

}

return root;

}